SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section introduces Bee2Bee, an AI-powered procurement workflow system designed to solve inefficiencies in SME restocking and supplier management. The system transforms fragmented and unstructured procurement inputs into structured, automated workflows, addressing the key problems identified in the Product Requirement Document.

1.1. Purpose:

This system analysis document highlights the technical scope, architectural design, and system-level decisions behind the development of Bee2Bee. It provides a comprehensive overview of how different components interact, how data flows through the system, and how AI is integrated as the central reasoning engine.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *Bee2Bee is designed as a cloud-based, workflow-driven system with a modular architecture. It consists of a frontend dashboard, backend API layer, workflow orchestration engine, AI reasoning layer, and database/storage services. Each component operates independently but is tightly integrated through API communication.*
2. ***Data Flows:***
    a. *Data flows from user interactions and external inputs (such as supplier messages and invoices) through the backend API, where it is processed and stored in the database. The system then passes relevant context to the AI layer for reasoning, and the results are fed back into the workflow engine to determine the next action.*
3. ***Model Process:***
    a. *The system model represents the end-to-end procurement workflow, starting from inventory monitoring and threshold analysis, followed by supplier negotiation, invoice processing, and final approval. Each stage is executed sequentially within a stateful workflow engine.*
4. ***Role of Reference:***
    a. *This document serves as a reference for developers and testers to understand the system architecture, workflow logic, and integration of AI components, ensuring consistency during development and testing.*

*1.2. **Background**:*

SMEs typically rely on manual procurement processes, where inventory monitoring, supplier communication, and financial reconciliation are handled separately. This leads to inefficiencies,

© UMHackathon 2026

delays, and increased risk of errors. Bee2Bee addresses this by introducing an integrated, AI-driven workflow system that automates these processes.

1. *Previous Version:*
    a. *Existing procurement systems are often fragmented and rely heavily on manual input. They lack intelligent decision-making capabilities and do not support automation of supplier communication or invoice validation. Most systems operate as simple dashboards without workflow orchestration or AI integration.*
2. *Changes in Major Architectural Components*
    The new system introduces several key capabilities, including AI-driven decision-making, a stateful workflow engine for multi-step orchestration, and the ability to process unstructured inputs such as messages, PDFs, and images. These enhancements significantly improve automation, scalability, and system intelligence.

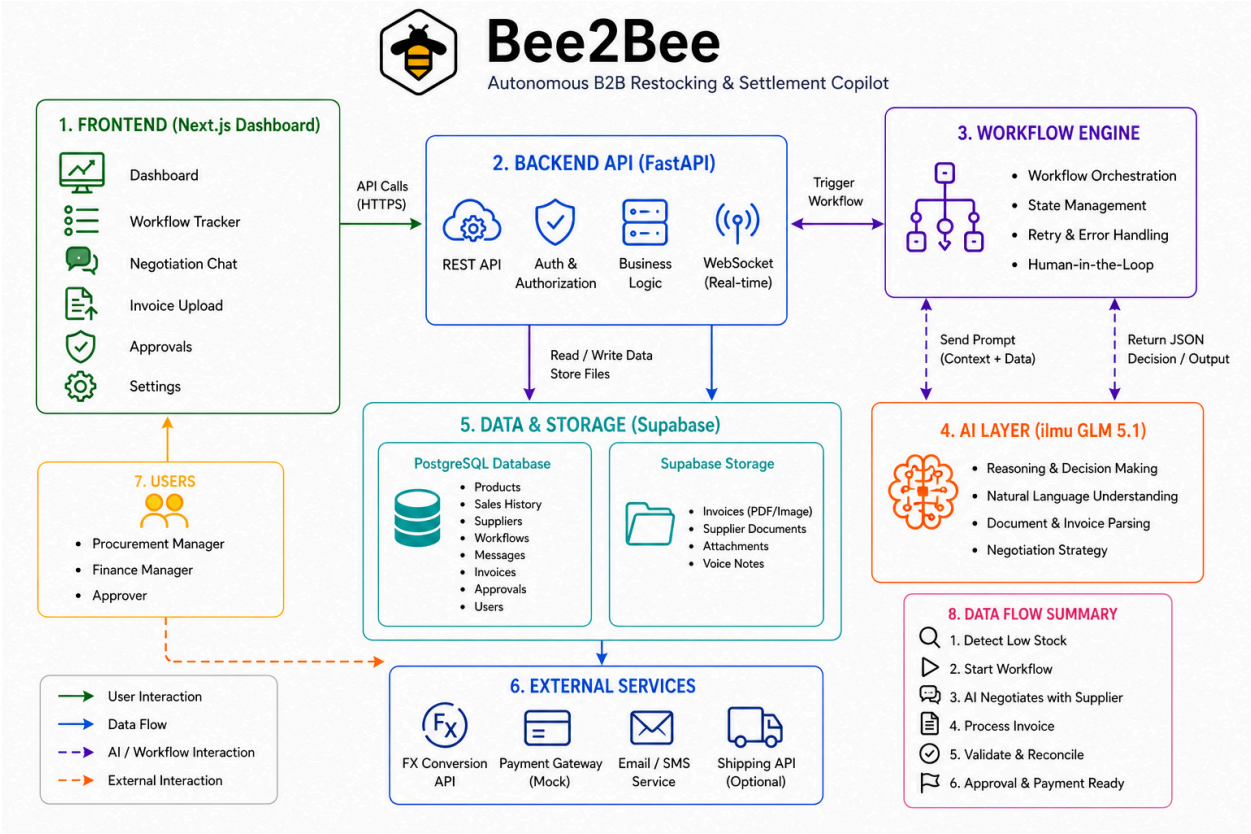
1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Customer (SME Merchants)	*Monitors inventory, reviews workflows, approves transactions*	*Clear workflow visibility, automation of procurement tasks, accurate recommendations*
Supplier	*Provides quotations, responds to negotiation messages*	*Clear communication, accurate order details*
Development Team	*Builds and maintains the system*	*Modular architecture, clear API contracts, scalable design*
QA Team	*Validating the system behavior for all the user roles*	*Defined workflows, clear edge cases, reliable outputs*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture



2.1.1. Overview:

Type	Details
System	*Web-based Dashboard (Frontend) + Backend Services*
Architecture	*Cloud-based, workflow-driven architecture with AI service layer*

Bee2Bee is structured as a client-server system where the frontend dashboard communicates with backend APIs. The backend integrates an AI service layer (ILMU GLM 5.1), and a database (Supabase/Postgres). The system is modular, with each component handling a specific responsibility such as data processing, workflow orchestration, or AI reasoning.

2.1.2 LLM as Service Layer

© UMHackathon 2026

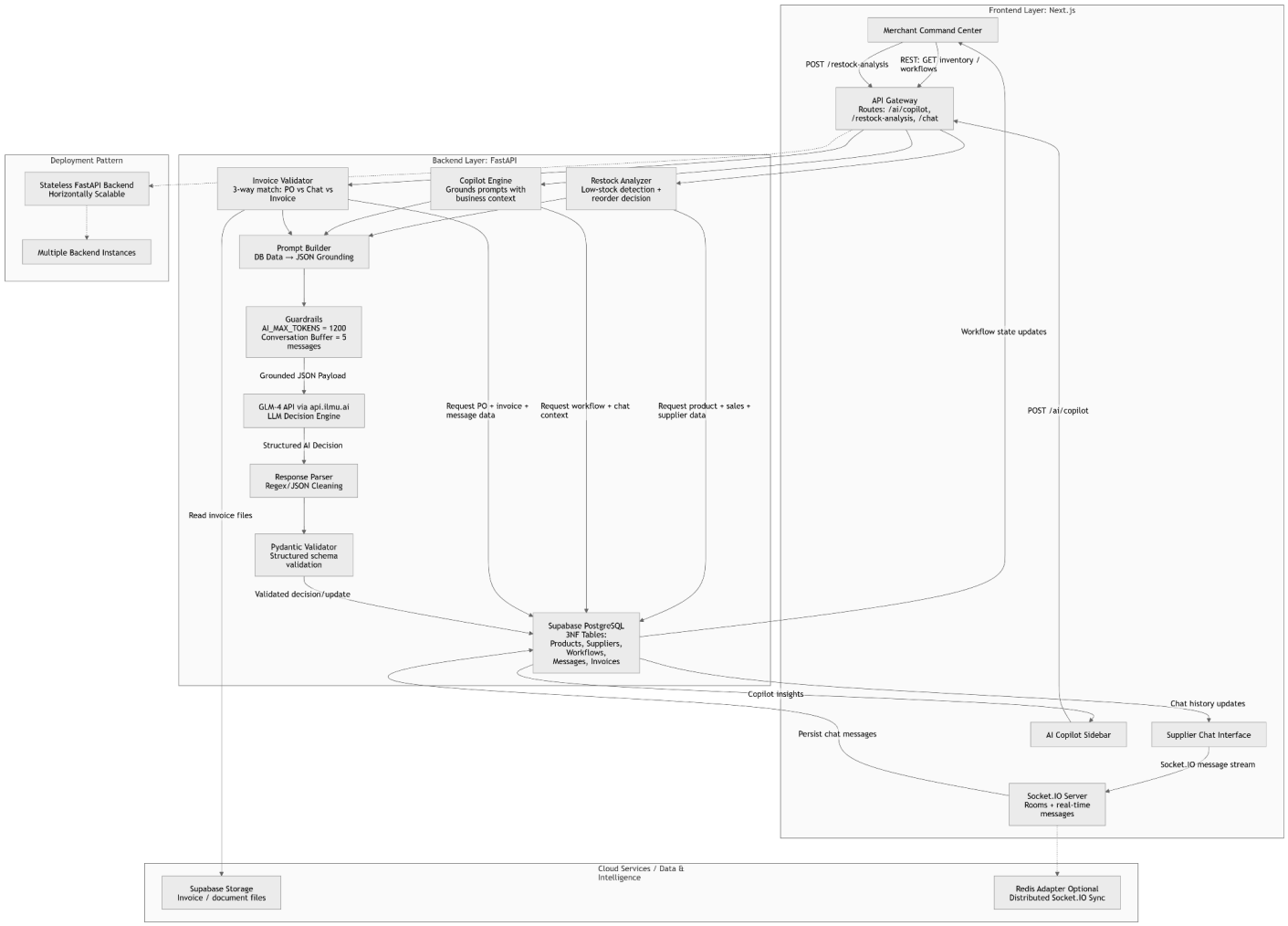
ILMU GLM 5.1 acts as the intelligence layer of Bee2Bee. Instead of hardcoding every business decision, the system sends structured business context to the LLM, such as stock levels, supplier information, invoice details, and restock history. The model then helps interpret the situation, generate recommendations, and return actionable outputs for the system to use.

ILMU GLM 5.1 is suitable because it is designed for low-latency and cost-efficient use cases, while still supporting reasoning capabilities through "thinking" features. This makes it practical for real-time workflows such as threshold analysis, restock suggestion, supplier negotiation drafting, invoice validation, and risk explanation.

In our architecture, the LLM does not replace the backend. The backend remains responsible for authentication, database operations, workflow state, and API control. *ILMU GLM 5.1* is called only when the system needs language understanding, reasoning, summarization, or decision support. This separation keeps the system reliable while still making the AI layer flexible and powerful.

© UMHackathon 2026

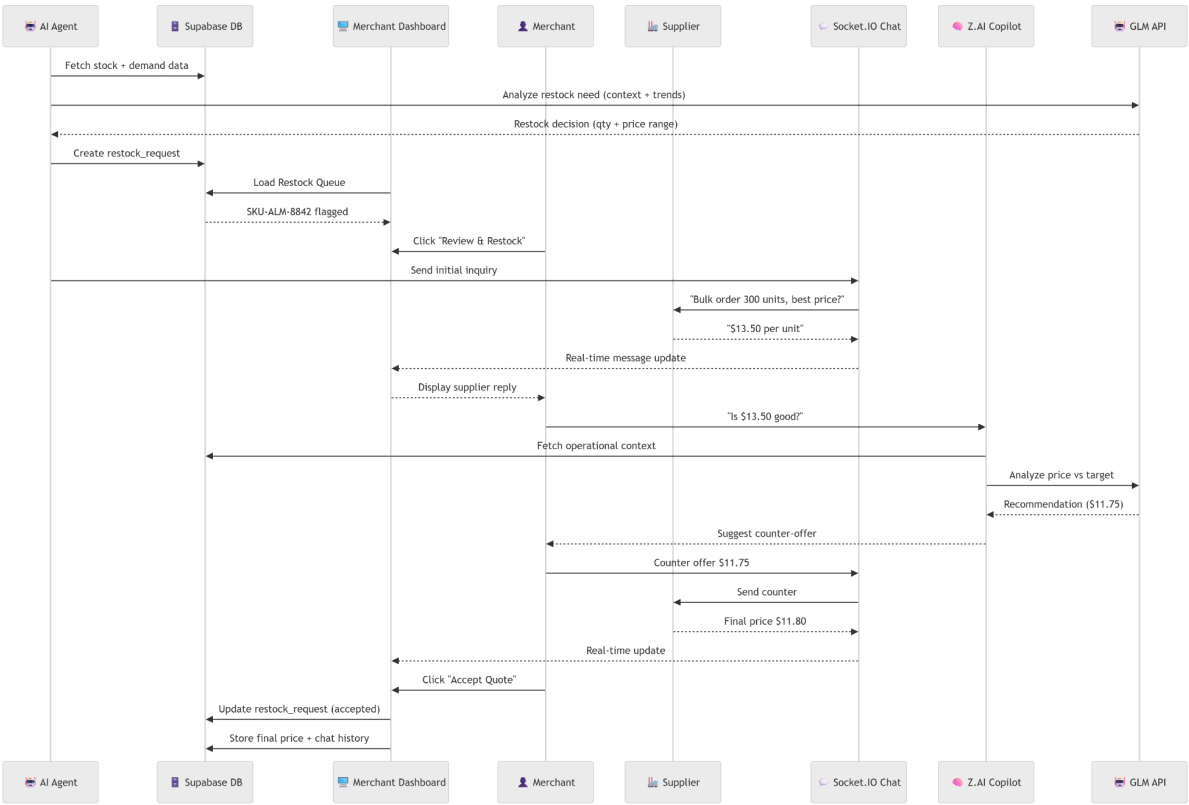
2.1.3 Dependency Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend - Next.js 15, TypeScript, Tailwind CSS, shadcn/ui

2.2.2. Backend - FastAPI – RESTful API for the microservices

2.2.3. Database - Supabase Postgres + Storage

2.2.4. AI Layer - ILMU GLM 5.1

2.2.5. Real-time - WebSockets/Supabase Realtime

2.2.6. External APIs - FX API / Mock settlement

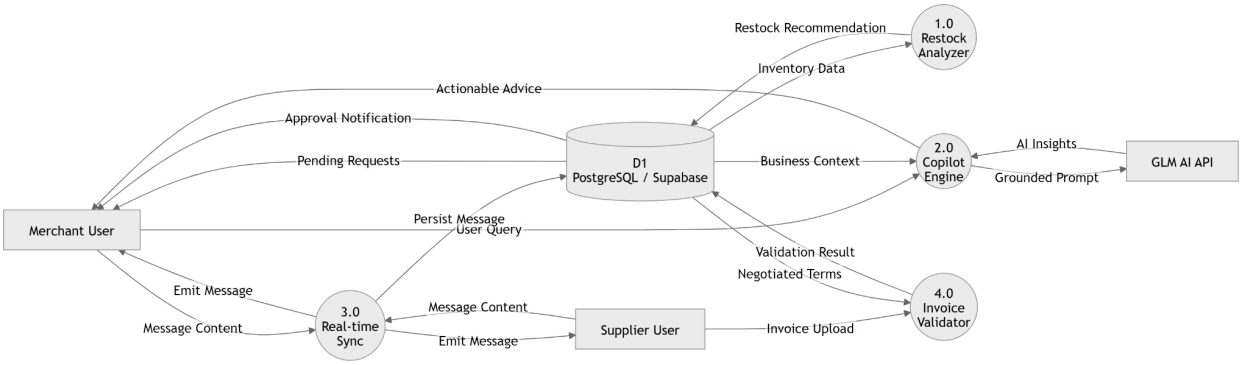
© UMHackathon 2026

Email: umhackathon@um.edu.my

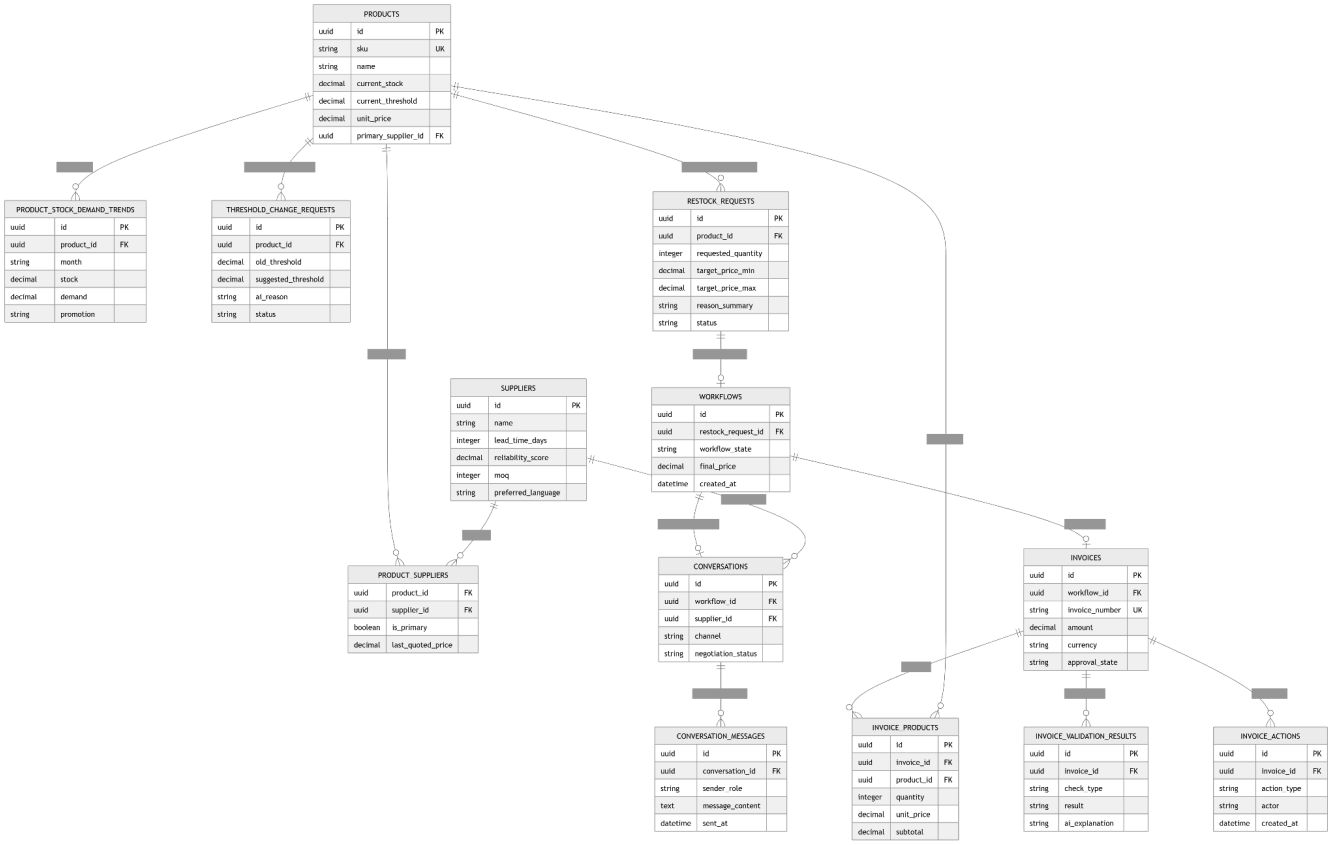
2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



a. **Normalized Database Schema**



3. Functional Requirements & Scope

© UMHackathon 2026

Email: umhackathon@um.edu.my

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.* ***[e.g. in Table]***

#	Features	Description
1	AI Inventory Prediction	Predicts demand and triggers restocking
2	Supplier Negotiation	AI handles communication and decision-making
3	Invoice Processing	Extracts and validates invoice data

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	System must handle multiple workflows simultaneously	Use modular backend + scalable services
Reliability	Workflow must not break mid-process	Temporal ensures retry and state persistence
Maintainability	Codebase should be modular	Separate frontend, backend, AI, workflow layers
Token Latency	AI responses within acceptable time	Async API calls + timeout handling
Cost Efficiency	LLM cost per session must be controlled	Preprocessing, chunking, caching

3.3. Out of Scope / Future Enhancements

a. Real payment gateway integration
b. Multi-user collaboration
c. Direct supplier system integration

© UMHackathon 2026

d. *Advanced analytics dashboard*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: *New features can be tested on limited workflows before full deployment to ensure stability.*

4.1.2. **Emergency Rollback (ER) & Golden Release**: *In case of system failure, workflows can revert to a stable version and fallback logic can be used.*

4.1.3. **Priority Matrix:**

4.1.3.1. **P1 Critical:** *Workflow failure or incorrect invoice validation → immediate rollback*

4.1.3.2. **P2 High:** *AI misinterpretation → escalate to user*

4.1.3.3. **P3 Medium:** *UI inconsistency → fix in next iteration*

4.2. Monitoring:

- *Track workflow states*
- *Monitor API response times*
- *Detect failures in AI responses*
- *Log all system actions*

4.3. Assumptions:

a. *Users have stable internet access*

b. *Supplier responses are available*

c. *AI API is accessible and responsive*

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Core reasoning engine	High dependency on API availability
Supabase	Database and storage	Medium risk if service downtime

5. Project Management & Team Contributions

© UMHackathon 2026

5.1. *Project Timeline:*

Day 1–2: Setup architecture and database
Day 3: AI integration and workflow engine
Day 4: Invoice and negotiation flow
Day 5: Testing and polishing

5.2. *Team Members Role:*

- *Benjamin Lim Shi Hern - temporal workflow + negotiation module*
- *Chen Wei Jay Nickolas - Frontend & Backend + invoice processing module*
- *Chu Cheng Qing - Data, Testing, Threshold Analysis module*
- *Lim Xin Rou - webSockets, Stock request modules*

5.3. *Recommendations:*

- *Use caching to reduce AI cost*
- *Implement retry logic for robustness*
- *Consider Redis for performance optimization*

© UMHackathon 2026